End-to-End Setup of Isaac Sim on AWS and Digital Twin Development

1. Introduction

The evolution of robotics, artificial intelligence, and simulation technologies has led to the development of high-fidelity digital environments capable of replicating real-world systems. One such advanced simulation platform is **NVIDIA Isaac Sim**, built on NVIDIA Omniverse, which enables realistic physics-based robotic simulations using GPU acceleration.

The objective of this work is to deploy Isaac Sim on a cloud-based infrastructure (AWS EC2) and enable remote access through graphical streaming and WebRTC protocols. This setup eliminates the need for local high-end hardware and allows scalable simulation environments.

2. Cloud-Based Simulation Concept

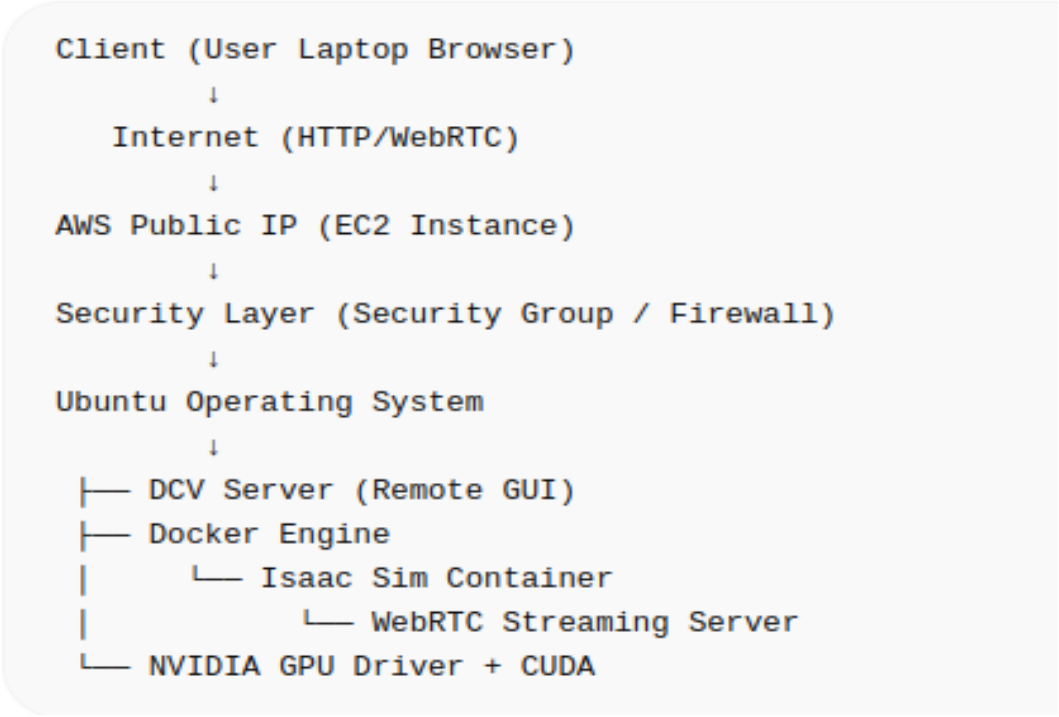
Traditionally, robotics simulations required powerful local machines equipped with GPUs. However, cloud computing introduces:

- **On-demand compute resources**
- **Scalability**
- **Remote accessibility**
- **Cost optimization (pay-per-use)**

AWS EC2 provides GPU-enabled instances that can run simulation workloads remotely, while the user interacts via a browser.

3. System Architecture (Conceptual)

The system operates on a **client-server architecture**:



4. GPU Computing in Simulation

The selected instance (**g6e.xlarge**) includes an NVIDIA L40S GPU.

Why GPU is essential:

- Parallel computation (thousands of cores)
- Real-time rendering (RTX ray tracing)
- Physics simulation acceleration
- AI model execution

Theoretical Insight:

CPU performs **sequential processing**, while GPU performs **massively parallel processing**, which is crucial for:

- Rendering multiple pixels simultaneously
- Computing physics interactions
- Processing sensor simulations

5. Containerization using Docker

Docker provides an abstraction layer that encapsulates:

- Application
- Dependencies
- Runtime environment

Why Docker is used:

- Avoids manual installation complexity
- Ensures environment consistency
- Allows portability across systems

In this setup:

Isaac Sim runs inside a Docker container, which interacts with the host GPU using NVIDIA Container Toolkit.

6. Networking & Security Model

AWS uses a **virtual firewall model** called a Security Group.

Key Concept:

Even if an application is running, it is inaccessible unless the port is explicitly allowed.

Ports used:

Port	Purpose
22	SSH access
8443	DCV GUI

8211 Isaac sim

7. Remote Visualization using DCV

DCV (Desktop Cloud Visualization) enables:

- Remote GPU-accelerated desktop access
- Low-latency rendering

Concept:

DCV acts as a **remote display protocol**, similar to RDP but optimized for GPU workloads.

8. Isaac Sim Execution Modes (Critical Theory)

Isaac Sim supports multiple execution modes:

Mode	Description
GUI Mode	Requires display (X11)
Headless Mode	No UI, backend processing
Streaming Mode	WebRTC-based remote UI

Problem Insight:

Running:

./runheadless.sh

→ Only backend runs

→ No UI available

Correct Mode:

./runheadless.native.sh --/app/livestream/enabled=true

→ Enables WebRTC streaming

9. WebRTC Streaming Mechanism

WebRTC (Web Real-Time Communication) is used to:

- Stream video frames from GPU
- Allow interactive control via browser

Working:

1. Isaac Sim renders frames using GPU
2. Frames are encoded
3. Sent via WebRTC server
4. Browser decodes and displays

Key Requirement:

- Port 8211 must be exposed
- Server must bind to public IP (`0.0.0.0`)

10. Error Analysis (Theoretical)

10.1 utmp Error:

[Failed to open /var/run/utmp]

Reason:
Containerized environments lack system-level user tracking.

Impact: None

10.2 rclpy Error:

[Could not import system rclpy]

Reason:
ROS2 not installed globally

Behavior:
Fallback to internal ROS

10.3 mdl_list_cache Warning

[mdl_list_cache is not complete]

Reason:
Asynchronous asset loading

Impact: Temporary delay

10.4 .Connection Reset Error

[curl: connection reset]

Reason:
Streaming service not initialized

10.5. No Windowing Error

[No windowing]

Reason:
Running GUI-dependent code without display server

11. Debugging Methodology

Effective debugging follows layered verification:

1. **Application Layer**
    - Is Isaac running?
2. **Container Layer**
    - Docker container status
3. **System Layer**
    - GPU availability
4. **Network Layer**
    - Port exposure
5. **Client Layer**
    - Browser access

12.Performance Considerations

Factors affecting performance:

- GPU utilization
- Network latency
- Initial shader compilation
- Asset loading time

First-run delay:

- Shader compilation
- Cache building
- Resource allocation

13.AWS Instance Creation (Concept + CLI)

Using AWS CLI

```bash
aws ec2 run-instances \
  --image-id ami-xxxxxxxx \
  --instance-type g6e.xlarge \
  --key-name isaac-key \
  --security-group-ids sg-xxxxxxxx \
  --subnet-id subnet-xxxxxxxx \
  --associate-public-ip-address
```

(Usually done via AWS Console UI)

14.Security Group Rules

Required Ports:

```
Port 22   → SSH
Port 8443 → DCV
Port 8211 → Isaac Sim Streaming
```

15. Connect to EC2

```bash
chmod 400 isaac-key.pem

ssh -i isaac-key.pem ubuntu@<PUBLIC-IP>
```

16. Install & Start DCV

```bash
sudo apt update
sudo apt install -y ubuntu-desktop

# Start display manager
sudo systemctl enable gdm3
sudo systemctl start gdm3

# Start DCV
sudo systemctl enable dcvserver
sudo systemctl start dcvserver
```

17.Create DCV Session

dcv create-session mysession – owner ubuntu

Access DCV

https://<PUBLIC-IP>:8443

18.Install Docker

```bash
sudo apt update
sudo apt install -y docker.io

sudo systemctl start docker
sudo systemctl enable docker

# Add user to docker group
sudo usermod -aG docker $USER
newgrp docker
```

19. Install NVIDIA Container Toolkit

```bash
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)

curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -

curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list \
  | sudo tee /etc/apt/sources.list.d/nvidia-docker.list

sudo apt update
sudo apt install -y nvidia-container-toolkit

sudo systemctl restart docker
```

20.Verify GPU

Nvidia-smi

21.Login to NVIDIA NGC

docker login nvcr.io

Username: $oauthtoken
Password: <NGC API KEY>

22.Pull Isaac Sim Image

docker pull nvcr.io/nvidia/isaac-sim:6.0.0-dev2

23.Run Isaac Sim

```bash
docker run --gpus all -it --rm \
-e ACCEPT_EULA=Y \
-e ENABLE_STREAMING=1 \
-e OMNI_SERVER_BIND=0.0.0.0 \
-p 8211:8211 \
nvcr.io/nvidia/isaac-sim:6.0.0-dev2 \
./runheadless.native.sh --/app/livestream/enabled=true
```

24. Access Isaac Sim UI

Open in **your laptop browser or system browser**

http://<PUBLIC-IP>:8211/streaming/webrtc-client

25.Debug Command

Check container running – docker ps

Check GPU usage – nvidia-smi
Check port – ss -tulpn | grep 8211
Test locally – curl http://localhost:8211
View logs – docker logs <container-id>

26.Common Errors & Fix Commands

Error: Port not accessible

Fix:

```
# Ensure docker port mapping
  -p 8211:8211
```

Error: Connection reset

Fix:

```
# Restart container with streaming flag
  --/app/livestream/enabled=true
```

Error: No windowing

Fix:

```
# Use correct script
  ./runheadless.native.sh
```

Docker issue:

```
sudo systemctl restart docker
```

Clean Docker:

```
docker system prune -f
```

27.Restart Full Setup

```bash
docker stop $(docker ps -q)
docker rm $(docker ps -aq)

sudo systemctl restart docker
```

28. Workflow Summary

Create EC2 → SSH → Install Docker → Setup GPU →

Login NGC → Pull Image → Run Container →

Open Browser → Access UI

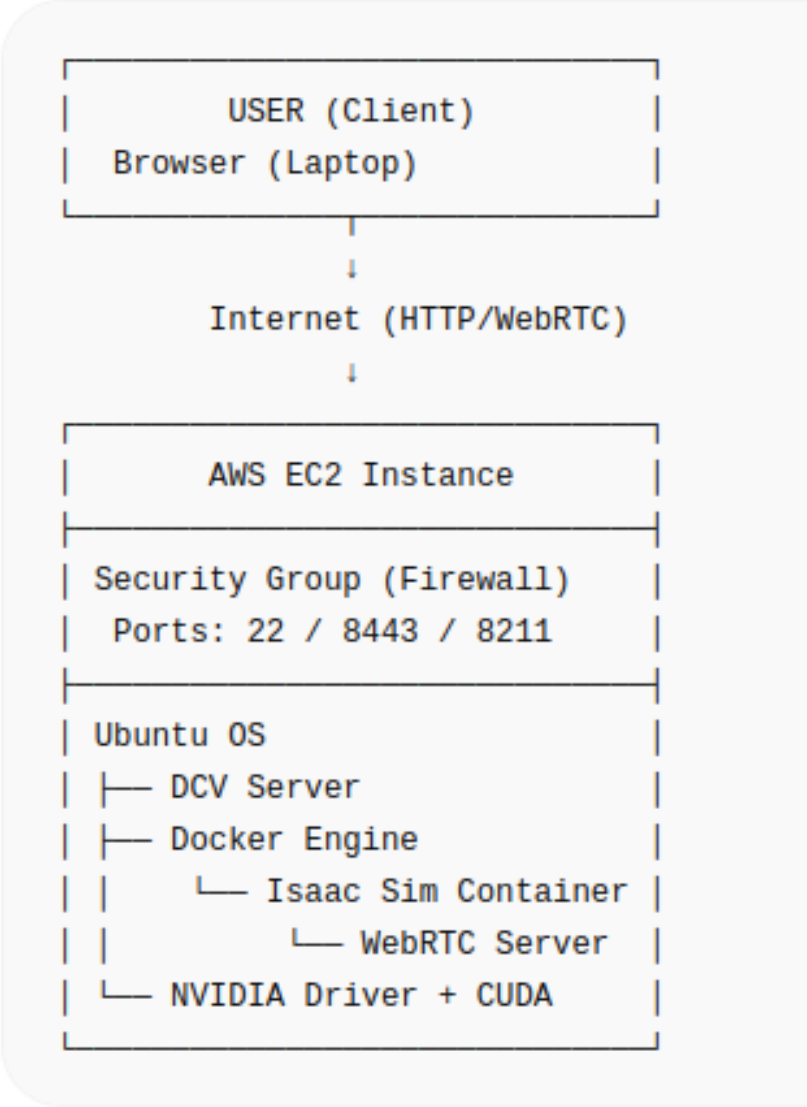
29.Linear Workflow Diagram

```
START
  ↓
Create AWS EC2 (GPU Instance)
  ↓
Configure Security Group (22, 8443, 8211)
  ↓
Connect via SSH / DCV
  ↓
Install Docker + NVIDIA Toolkit
  ↓
Verify GPU (nvidia-smi)
  ↓
Login to NGC
  ↓
Pull Isaac Sim Image
  ↓
Run Docker Container (Streaming Mode)
  ↓
Expose Port 8211
  ↓
Open Browser (Local PC)
  ↓
Access Isaac Sim UI
  ↓
END
```

Step-by-step execution

👉 Focus: **What you did in order**

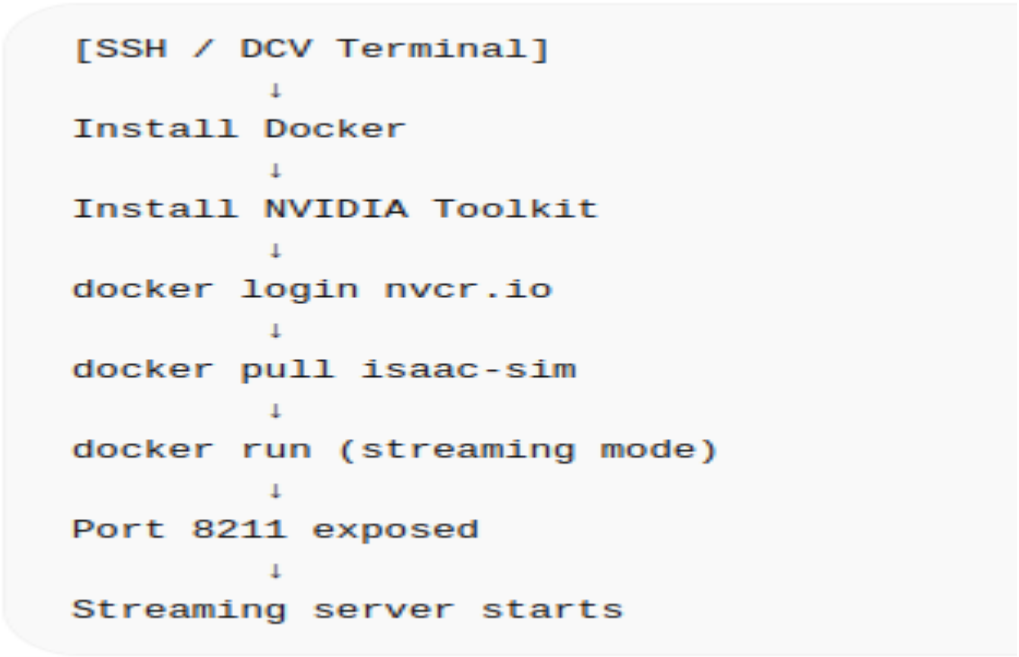
30.Layered Architecture Diagram



System structure

👉 Focus: **How components are arranged**

31.Command Execution Flow Diagram



Terminal actions

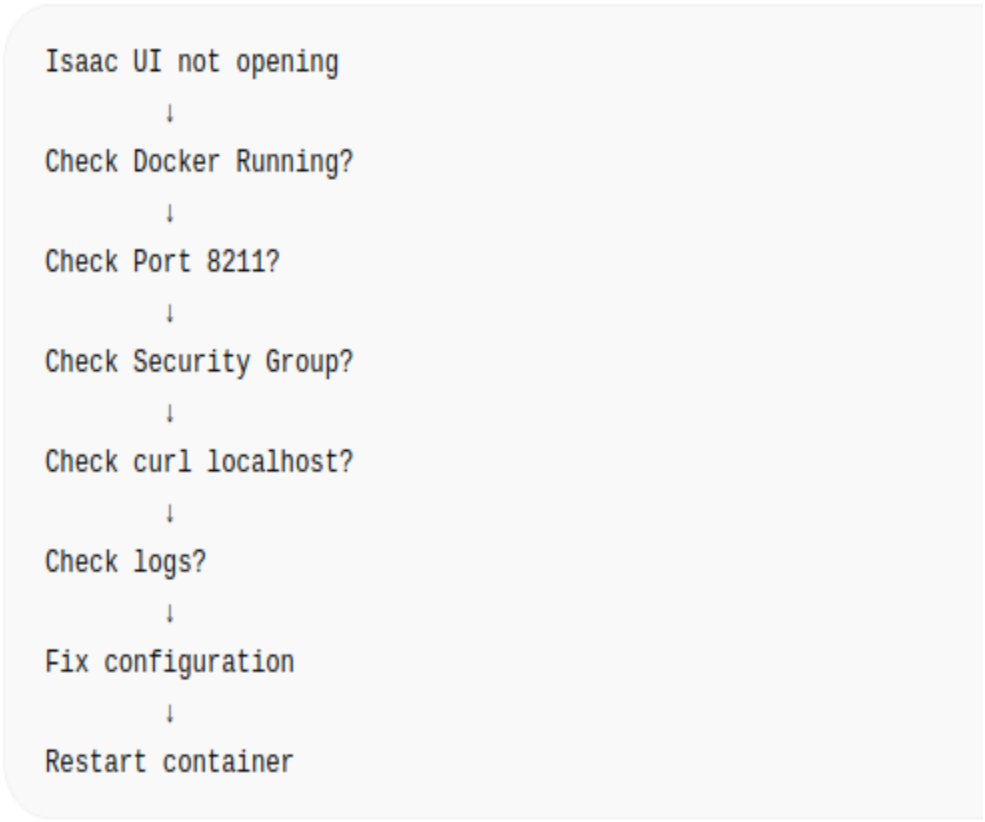
👉 Focus: **What commands you ran**

32.Networking Flow Diagram

Data movement

👉 Focus: **How browser connects to server**

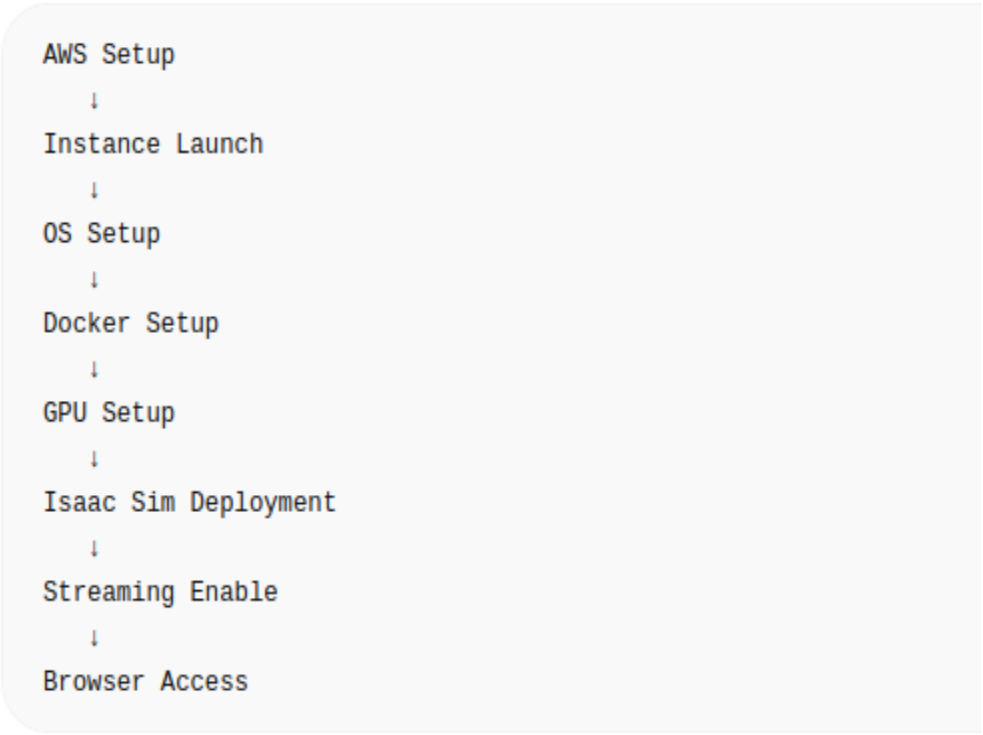
33.Debugging Flow Diagram



Problem solving path

👉 Focus: **How you fixed errors**

34.Deployment Pipeline Diagram



Setup stages

👉 Focus: **Installation phases**

35.Conceptual Flow

```
Cloud Infrastructure
        ↓
GPU Compute Layer
        ↓
Containerization (Docker)
        ↓
Simulation Engine (Isaac Sim)
        ↓
Streaming Layer (WebRTC)
        ↓
Client Visualization (Browser)
```

Theory explanation

👉 **Focus: Understanding system logically**

DIGITAL TWIN

What is a Digital Twin?

A digital twin is a live, virtual replica of a real-world system—such as a robot, machine, building, or entire facility—that stays synchronized with its physical counterpart using data. It doesn't just look like the real system; it behaves like it, because it uses the same geometry, physics, and (often) real sensor data.

1. Folder Structure:

```
mkdir -p ~/isaac_ws/{robots,worlds,scripts}

cd ~/isaac_ws
```

2. Import Robot (URDF → Isaac)

Place your URDF: cp your_robot.urdf ~/isaac_ws/robots/

3.Convert URDF to USD (inside Isaac container)

Run Isaac container:

```
docker run --gpus all -it --rm \
nvcr.io/nvidia/isaac-sim:6.0.0-dev2
```

Inside container:

```
cd /isaac-sim
./python.sh
```

4.Python Script (URDF Import):

```
from omni.isaac.kit import SimulationApp
simulation_app = SimulationApp({"headless": False})
from omni.isaac.core.utils.stage import open_stage
from omni.isaac.core.utils.prims import create_prim
from omni.isaac.urdf import _urdf
# Load URDF
urdf_path = "/home/ubuntu/isaac_ws/robots/your_robot.urdf"
_urdf.import_urdf(
    file_path=urdf_path,
    prim_path="/World/Robot",
    fix_base=False
)simulation_app.update()
```

👉 Save as:

```
nano ~/isaac_ws/scripts/import_robot.py
```

Run:

./python.sh ~/isaac_ws/scripts/import_robot.py

5.Load Environment (Digital Twin World)

If you have `.usd`:

```
from omni.isaac.core.utils.stage import open_stage
open_stage("/home/ubuntu/isaac_ws/worlds/hospital.usd")
```

4. Add Robot to Environment

```
from omni.isaac.core import World
from omni.isaac.core.utils.prims import create_prim
world = World()
create_prim(
      "/World/Robot",
       "Xform",
        position=(0, 0, 0)
)
world.reset()
```

5.Add Sensors (Camera / LiDAR)

CAMERA:

```
from omni.isaac.sensor import Camera
camera = Camera(
prim_path="/World/Robot/camera",
position=(0.5, 0, 1.0),
frequency=30
```

```
    )
   camera.initialize()
```

LIDAR:

```
from omni.isaac.sensor import LidarRtx
lidar = LidarRtx(
      prim_path="/World/Robot/lidar",
      config="Velodyne_VLP_16"
)
lidar.initialize()
```

6.Run Simulation Loop

```
    from omni.isaac.core import World
    world = World()
    while simulation_app.is_running():
        world.step(render=True)
```

7.Connect ROS2 (Digital Twin ↔ Real Robot)

Inside container: export ROS_DOMAIN_ID=0

Enable ROS bridge in script:

```
from omni.isaac.ros2_bridge import ROS2Bridge
bridge = ROS2Bridge()
```

Run ROS2 topic (outside): ros2 topic list

8.Run Full Digital Twin via Docker

```
docker run --gpus all -it --rm \

-e ACCEPT_EULA=Y \

-e ENABLE_STREAMING=1 \

-e OMNI_SERVER_BIND=0.0.0.0 \

-p 8211:8211 \

nvcr.io/nvidia/isaac-sim:6.0.0-dev2 \

./runheadless.native.sh --/app/livestream/enabled=true
```

9. Open UI

http://<YOUR-IP>:8211/streaming/webrtc-client

10. Debug Commands

```
docker ps

nvidia-smi

ss -tulpn | grep 8211
```

Workflow:

Real Robot → Sensors → Data → Isaac Sim

↓

Virtual Robot (Twin)

↓

Simulation / Testing / AI

Overall conclusion:

The overall workflow for deploying NVIDIA Isaac Sim on an AWS GPU instance demonstrates the successful integration of cloud computing, GPU acceleration, containerization, and real-time streaming technologies.

The setup process involved multiple stages, including instance provisioning, network configuration, remote access setup using DCV, Docker-based deployment, and enabling WebRTC streaming for browser-based visualization. Each stage highlighted the importance of correct configuration, particularly in areas such as security groups, port exposure, container execution modes, and client-server interaction.

Throughout the workflow, several challenges were encountered, including incorrect Docker image selection, networking restrictions, port accessibility issues, and misunderstandings of execution modes. These issues emphasized that most failures in such deployments are not due to the simulation software itself, but rather due to misconfigurations in infrastructure, networking, and environment setup.

By systematically debugging each layer—ranging from container status and GPU utilization to network accessibility and browser connectivity—the system was successfully brought to a fully functional state. The final setup enables remote access to a high-performance simulation environment directly from a local browser, eliminating the need for local GPU resources.

This workflow illustrates that deploying advanced simulation platforms in the cloud requires not only technical execution but also a clear understanding of system architecture, networking principles, and distributed computing concepts. Once properly configured, the system provides a scalable, efficient, and flexible environment for robotics simulation, digital twin development, and future AI-based applications.

A **digital twin** is a high-fidelity virtual representation of a real-world system that mirrors its structure, behavior, and performance in a simulated environment. Using platforms like NVIDIA Isaac Sim, a robot and its surrounding environment can be recreated with realistic physics, sensors, and interactions. This allows developers to test operations, analyze system behavior, and simulate real-world scenarios without physically deploying the robot, thereby reducing risk, cost, and development time.

In practical applications, a digital twin enables continuous improvement by allowing safe experimentation, optimization of workflows, and predictive analysis. When integrated

with real-time data or systems like ROS2, it can closely synchronize with the physical system, providing valuable insights and control capabilities. Overall, digital twins play a crucial role in modern robotics and automation by bridging the gap between simulation and reality, enabling smarter and more efficient system design.
